

Working with Very Large Datasets

Nowadays, people often have to work with very large datasets.

Especially if you want to train a transformer model like **BERT** or **GPT-2** from the beginning, the dataset can become very large.

The Problem with Large Datasets

The problem is:

- the dataset can be several GB or even hundreds of GB
- it is hard to load that much data into RAM at once
- a laptop or desktop may freeze or hang

For example, the **WebText dataset** used to train GPT-2 contained:

- more than **8 million documents**
- about **40 GB of text**

If you try to load that much data into your laptop's RAM all at once, it can create serious problems.

How the Datasets Library Helps

The **Datasets** library solves this problem in two main ways.

1) Memory Mapping

This means the whole dataset does **not need to be loaded into RAM**.

The dataset stays on disk, and only the required parts are accessed when needed.

2) Streaming

Streaming means the data is **loaded only when needed**.

You do not need to download the entire dataset before using it.

Which Dataset Is Discussed in This Section?

In this section, they discuss a very large dataset called **The Pile**.

What Is The Pile?

The Pile is a **large English text corpus** created by **EleutherAI** to train large language models.

It includes many different types of data, such as:

- scientific articles
- GitHub code repositories
- filtered web text

The total size of the dataset is about **825 GB**, which is extremely large.

Which Subset Is Used as an Example?

They demonstrate the **PubMed Abstracts dataset**.

This dataset contains:

- abstracts from **biomedical publications on PubMed**
- around **15 million research papers**

So each entry is basically the **summary text of a medical research paper**.

Why Install **zstandard** Before Loading the Dataset?

```
!pip install zstandard
```

The dataset file is compressed using the **.zst format**.

To decompress and read this file, the **zstandard library** is required.

How the Dataset Is Loaded

```
from datasets import load_dataset
```

```
data_files =  
"https://the-eye.eu/public/AI/pile_preliminary_components/PUBMED_title_abstracts_2019_base/line.jsonl.zst"  
  
pubmed_dataset = load_dataset(  
    "json",  
    data_files=data_files,  
    split="train"  
)
```

What happens here:

- the file is loaded from a **remote URL**
 - the file type is **JSON**
 - the **train split** of the dataset is loaded
-

What the Output Shows

```
Dataset(  
  features: ['meta', 'text'],  
  num_rows: 15518009  
)
```

This means:

The dataset has **two columns**:

- **meta**
- **text**

And the total number of rows is:

15,518,009

So this is a **very large dataset**.

What Does the First Example Look Like?

pubmed_dataset[0]

From this example we can see:

- the **meta field** contains information about the article
 - for example **pmid**
 - and the language
- the **text field** contains the **abstract of the article**

So each row contains the **abstract of a medical research paper**.

The Interesting Part: Why Is RAM Usage Low?

They used the **psutil library** to check RAM usage.

First install the library:

```
!pip install psutil
```

Then measure the memory usage:

```
import psutil
```

```
print(  
    f"RAM used: {psutil.Process().memory_info().rss / (1024 * 1024):.2f} MB"  
)
```

Here, **rss** means the amount of RAM used by the process.

The output was approximately:

RAM used: 5678.33 MB

So about **5.7 GB of RAM** was used.

But the Dataset Size Is Much Larger

Next they checked the actual dataset size:

```
print(f"Dataset size in bytes: {pubmed_dataset.dataset_size}")
```

```
size_gb = pubmed_dataset.dataset_size / (1024**3)
```

```
print(f"Dataset size (cache file): {size_gb:.2f} GB")
```

The output showed:

- dataset size $\approx$ **19.54 GB**

So:

- the dataset on disk is about **20 GB**
- but RAM usage is only about **5.7 GB**

This is one of the **major advantages of the Datasets library**.

How Is This Possible?

This works because of **memory mapping**.

What Is Memory Mapping?

In simple words:

Instead of loading the entire dataset into RAM,

- the dataset stays **stored on disk**
- the program **accesses small parts only when needed**

It is similar to reading a book:

You do not memorize the whole book.
You just open the page you need.

Because of this, RAM usage becomes much smaller.

Difference Between Pandas and Datasets

With **Pandas**, large datasets often require a lot of RAM.

Sometimes you may need **5 to 10 times more RAM than the dataset size**.

However, **Datasets** uses memory-mapped files, so:

- the entire dataset is not loaded into RAM
 - only required parts are accessed
 - RAM usage remains much lower
-

The Role of Apache Arrow

The high speed and efficiency come mainly from:

- **Apache Arrow**
- **pyarrow**

These technologies store data in a format that:

- allows very fast reading
 - uses less memory
 - makes data processing faster
-

Another Big Advantage: Parallel Processing

Memory-mapped files can be shared across multiple processes.

This means:

- `Dataset.map()` can run in **parallel**
 - the dataset does **not need to be copied**
 - processing becomes faster
-

They Also Performed a Speed Test

They iterated over the dataset in batches:

```
import timeit
```

```
code_snippet = """batch_size = 1000
```

```
for idx in range(0, len(pubmed_dataset), batch_size):
    _ = pubmed_dataset[idx:idx + batch_size]
"""
```

Here:

- 1000 rows are processed at a time
- the loop goes through the entire dataset

Then they measured the time using `timeit`.

The output was:

Iterated over 15518009 examples (about 19.5 GB) in 64.2s, i.e. 0.304 GB/s

This means:

- reading more than **15 million examples** took about **64.2 seconds**
- the speed was about **0.304 GB per second**

This is a very good speed.

What Problems Can Still Occur?

Even with these improvements, there may still be situations where:

- the dataset does not fit into RAM
- the hard drive does not have enough space

For example, downloading the entire **Pile dataset** would require about **825 GB of free disk space**.

Most laptops do not have that much space.

Below is your content translated into **simple English** in the **same style**, with **large bold headings**.

Solution to This Problem: Streaming

At this point, the **Datasets** library provides a **streaming** feature.

What Is a Streaming Dataset?

Normally, when you load a dataset, the library prepares the dataset on your local system so that you can use it.

But if the dataset is very large, then:

- it takes time to download
- it needs a lot of disk space
- it becomes hard to keep the whole dataset

To solve this problem, **Datasets** provides **streaming**.

The Idea of Streaming

Instead of downloading the whole dataset first, it brings in only the data that is needed at that moment.

It is just like video streaming:

- you do not download the whole movie first
- only the part you are watching comes in

The same idea works for datasets too.

1) How Do You Turn On Streaming?

You just need to pass `streaming=True` inside `load_dataset()`.

```
pubmed_dataset_streamed = load_dataset(  
    "json",  
    data_files=data_files,  
    split="train",  
    streaming=True  
)
```

Here, the dataset is being loaded in streaming mode.

2) What Type of Object Do We Get?

Earlier, we used to get a `Dataset` object.

But when we use `streaming=True`, we get an `IterableDataset`.

What Does IterableDataset Mean?

It means you have to access the dataset **one item at a time by iterating over it**.

That means:

- you cannot always use normal indexing easily like `dataset[0]`
- instead, you need to use a loop or an iterator to get the data

3) How Do We Get the First Element?

```
next(iter(pubmed_dataset_streamed))
```

Here:

- `iter(...)` turns the dataset into an iterator
- `next(...)` gives the first example

So this is how we get the first row from the streamed dataset.

4) Can We Process a Streaming Dataset?

Yes, we can.

We can use `IterableDataset.map()` for **on-the-fly processing**.

For example, they used it for tokenization:

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
tokenized_dataset = pubmed_dataset_streamed.map(lambda x: tokenizer(x["text"]))
next(iter(tokenized_dataset))
```

What is happening here:

- the `text` field is taken
- the tokenizer converts the text into tokens
- the output contains `input_ids` and `attention_mask`

In Simple Words

The dataset is coming in, and it is being processed at the same time.

This means you do not need to tokenize and save the whole dataset first.

5) Why Is `batched=True` Useful?

To make tokenization faster in streaming mode, you can use `batched=True`.

That means:

- instead of processing 1 example at a time
- it processes a whole batch of examples together

This increases speed.

The default batch size is usually **1000**.

6) Can a Streamed Dataset Be Shuffled?

Yes, it can.

But normal `Dataset.shuffle()` and streamed `IterableDataset.shuffle()` do not work in the same way.

```
shuffled_dataset = pubmed_dataset_streamed.shuffle(buffer_size=10_000, seed=42)
```

Here, the whole dataset is not shuffled at once.

Only the data inside a **buffer** is shuffled.

What Does Buffer Mean?

Here, `buffer_size=10000` means:

- 10,000 examples are kept in memory at one time
- random shuffling happens only inside those 10,000 examples

Then:

- when one example is taken out
- the next example from the dataset comes in and fills that place

In Simple Words

Instead of shaking the whole river, you are only mixing water inside one bucket.

7) What Does `take()` Do?

```
dataset_head = pubmed_dataset_streamed.take(5)
list(dataset_head)
```

This takes the **first 5 examples** from the dataset.

`take(5)` means we want the first 5 rows.

Then `list(...)` shows them together.

8) What Does `skip()` Do?

`skip(n)` means it skips the first `n` examples and takes the rest.

Example:

```
train_dataset = shuffled_dataset.skip(1000)
validation_dataset = shuffled_dataset.take(1000)
```

Here:

- the first 1000 examples go to validation
- the rest go to training

In Simple Words

We keep the first 1000 separately for validation, and the remaining part becomes the training set.

9) Why Do We Need `take()` and `skip()`?

Because in a streamed dataset, normal `select()` is not always convenient.

So:

- `take()` means take some examples from the front
- `skip()` means leave some examples from the front

These make it easy to create splits.

10) Can We Combine Multiple Streamed Datasets?

Yes, we can.

For that, we use `interleave_datasets()`.

It is very useful when you want to combine many large datasets into one big corpus.

11) What Is the FreeLaw Dataset?

As another example, they used another large dataset:

FreeLaw Opinions

This dataset contains:

- legal opinions from US courts
- about **51 GB** of data

It was also loaded in streaming mode:

```
law_dataset_streamed = load_dataset(
    "json",
```

```
data_files="https://the-eye.eu/public/AI/pile_preliminary_components/FreeLaw_Opinions.jsonl.z
st",
    split="train",
    streaming=True,
)
```

That means:

- the dataset is large
- still, it can be accessed easily in streaming mode
- RAM usage stays low

12) What Does `interleave_datasets()` Do?

```
from datasets import interleave_datasets

combined_dataset = interleave_datasets([
    pubmed_dataset_streamed,
    law_dataset_streamed
])
```

This combines two streamed datasets into one new dataset.

How Does It Combine Them?

It combines them in an **alternating** way.

That means:

- 1st example → from PubMed
- 2nd example → from FreeLaw
- 3rd example → from PubMed
- 4th example → from FreeLaw

And it keeps going like this.

In Simple Words

It is like taking one person from one line, then one person from another line, one after another.

13) Why Did They Use `islice()`?

```
from itertools import islice

list(islice(combined_dataset, 2))
```

This is used to see the **first 2 examples** from the combined dataset.

Because a streamed dataset is iterator-based, `islice()` is a convenient way to view a few samples.

14) Can the Whole Pile Also Be Streamed?

Yes.

They gave an example of streaming the full **825 GB Pile dataset**.

```
base_url = "https://the-eye.eu/public/AI/pile/"
data_files = {
    "train": [base_url + "train/" + f"{idx:02d}.jsonl.zst" for idx in range(30)],
    "validation": base_url + "val.jsonl.zst",
    "test": base_url + "test.jsonl.zst",
}

pile_dataset = load_dataset("json", data_files=data_files, streaming=True)
```

Here:

- many files are given for the train split
- validation and test have separate files
- everything is loaded in streaming mode

So the full 825 GB dataset does not need to be downloaded at once.

15) What Are the Biggest Advantages?

The main advantages of streaming are:

Less Disk Space Is Needed

You do not need to save the whole dataset first.

Less RAM Is Needed

Only the needed data comes in.

Huge Datasets Can Be Handled

You can work with a large corpus even on your own laptop.

Useful During Training

It is especially useful when the dataset is very large.

16) Difference Between Normal Dataset and Streaming Dataset

Topic	Normal Dataset	Streaming Dataset
Object type	<code>Dataset</code>	<code>IterableDataset</code>
Access	indexing / slicing	iteration
Full data download	usually needed	not needed
For huge datasets	may be limited	very useful
Shuffle	full shuffle possible	buffer-based shuffle

Very Simple Summary

`streaming=True`

It brings data as needed instead of downloading the whole dataset first.

`IterableDataset`

You read it using a loop or iterator.

`map()`

You can process the data as it comes in.

`shuffle(buffer_size=...)`

It does not shuffle the whole dataset, only the data inside the buffer.

take(n)

It takes the first `n` examples.

skip(n)

It skips the first `n` examples.

interleave_datasets()

It combines multiple datasets by taking examples from them one after another.